HKDSE ICT SBA Gomoku Project Technical Document

Made by Jimmy Yang.

Section 1 : Introduction

This is a technical document for my school-based assessment Gomoku project. The program is based on *Python 3.14.3* interpreter while can be executed under *Windows Terminal* and *macOS/Linux terminal environment* with the support of *Python (>=3.14.3)* interpreter.

The program's core function is to spawn a 15×15 standard Gomoku chessboard and supports the coordinates method for arranging the chesses. Other basic functions are automatic winner recognition and the refresh of the terminal screen.

Program's development environment is standard *Python 3.14.3* under *Arch Linux x86_64*, while doesn't have any requirements of external third-party libraries.

Section 2 : Data & Variables

In this section, I will explain how do I store the game's status.

Board's presentation: use 2-dimensional array to initialize the board as empty. The reason for choosing the 2-dimensional is the easy access of the data through `board[row][col]`.

The accessing code example below is from the *winner recognition* part.

```
while 0 <= nr < SIZE and 0 <= nc < SIZE and board[nr][nc] == p:
```

Status variables: In the code, I set 2 main global variables for the game's experience optimization, `current_player` is set as a storage of current placing player, used for the judgment and the message system.
`status_msg` is set as a temporary stack for the game's status, and printed out when a event is happened during the game's process. The reason we need a temporary stack is that we need a variable to store the status message even after the terminal screen has been cleared. After the clear, the support of `status_msg` will provide us the message used for displaying on the next game frame.

Section 3 : Module Composition

In this section, I will explain how my program runs, and its core algorithms, while having some quotes of the code.

Section 3 Part 1 : Parsing Input

The first part will be the method of breaking user's input into parts that the program can understand. In this part, the user will type in their potential coordinates into the terminal like A8 or H8, however, the algorithms cannot directly decide the position, so we need to break it apart, and transfer them into offsets to the array.

```
def parse_input(input_str):
    s = input_str.strip().upper()
    if s == 'Q':
        return 'QUIT', ""
```

This part of the code will first accept the user's input, and check if the user's input is requiring the program to interrupt and exit, by checking if the input is the exit signal `"Q"`.

```
    if len(s) < 2:
        return None, "Coordinate too short."
```

This part of the code will secondly accept the user's input, if the user's request is not interrupting and quitting the program. It will check the input's length, and make sure the input is long enough to proceed, otherwise it will return `None` and require user to re-enter the coordinates.

```
    try:
        col = ord(s[0]) - ord('A')
        row = int(s[1:]) - 1

        if 0 <= col < SIZE and 0 <= row < SIZE:
            return (row, col), ""
        else:
            return None, "Coordinate out of range."
    except (ValueError, IndexError):
        return None, "Invalid input."
```

This part of the code is the error handling part. It uses the `try catch throw` method of catching and handling error, whether to jump to another part of the program or just simply interrupt the program from continue running, and pass the further processing to end-user. The `try catch throw` method has several benefits than traditional error handling:

1. Code is clear and easy to read, rather than bunch of `if-else`, benefit the future extend and maintenance.
2. `try-except` can catch runtime errors easily without interrupting the entire program and pops up a error message.
3. The usage of this error handling method has performed a famous thinking in *Python* world, that is *EAFP*, *Easier to Ask for Forgiveness than Permission*.

If the user is trying to input values that is out of the range, it will return a `None` and then told the user to input again, but not interrupt the program itself.

Section 3 Part 2 : Clearing Screen

Tired of scrolling down the terminal page? Then `os` module have you covered.

```
def clear_screen():
    os.system('cls' if os.name == 'nt' else 'clear')
```

This part of the code takes a very little of the total space of the entire script, but it improves the experience of the game a lot. Rather than keep scrolling down the terminal output, it will automatically clear the screen for you, keep your mind focusing on the chess board.

The core logic of the code is checking OS type and then send the correct clearing command to the terminal emulator. If the result of `os.name` is `nt`, which is a abbreviation of a certain type of OS `Windows NT`, or more commonly the normal *Windows* we use today. If the result was not `nt`, then the program will recognize the environment as `POSIX Compliant System`, or more commonly *macOS*, *UNIX* or *Linux*.

Section 3 Part 3 : Creating Board

This section of the code is simply create a two-dimensional array which is used for storing the data of the chess board.

```
def create_board():
    return [[EMPTY for _ in range(SIZE)] for _ in range(SIZE)]
```

Section 3 Part 4 : Printing Board

In this part of the program, the program itself will print or display the board to the current terminal screen.

Below is the first part of the code, it will first define the function itself, which let itself can be used in the main program, then it will call `clear_screen` function, it was used for clearing the terminal output to keep the screen clean, in order to improve the game experience.

```
def print_board(board, status_msg, current_player):
    clear_screen()
```

After clearing the screen, program will first discover the 15 letters which is board is using, and then print the board by adding line break symbol (`\n`) and spaces. Then it will use `-` as a symbol for printing the border of the chess board.

```
    letters = [chr(ord('A') + i) for i in range(SIZE)]
    print("\n     " + " ".join(letters))
    print("    +" + "--" * SIZE + "+")
```

In the next period of the code, the script will call `for` loop, use `i` as a local variable, and the global variable `SIZE` as the range for the loop. Then it will set `row_num` and `row_content`, these two local variable will help the script to adjust the length of theselection of data types and data structures

⚫

variable/constant

declaration and initiali s ation row, or in another way, to make it aligned. After that, `row_content` will help us to store the chess' position data in the two-dimensional array we created. Moreover, the `print` function will print the position indicator (A5 or H6) above the border.

```
for i in range(SIZE):
    row_num = str(i + 1).rjust(2)
    row_content = " ".join(board[i])
    print(f" {row_num} | {row_content} | {row_num}")
```

Behind this period of the code, the script will call `print` function for 2 times, in order to display the bottom border of the chess board.

```
print("    +" + "--" * SIZE + "+")
print("     " + " ".join(letters))
```

Additionally, the script will call `print` function for 3 times, in order to print `Current Player`, `System Msg\(abbreviation of message\)` and the bottom of the board. This is one of the most core part of the script, because it provides a improved experience for the player, and a pipeline for the player to know the situation and the event, or even a error thrown. We need to include this part of `print` function block in the `print_board` part, in order to keep them on the screen after we call the `clear_screen` function to clear the screen every time when the board needs refreshing.

```
print(f"\n[ Current Player ]: {current_player}")
print(f"[ System Msg ]: {status_msg}")
print("-" * 40)
```

Section 3 Part 5 : Winner Checking

In this part of the technical document, we will break apart the most core algorithms in the Gomoku script - the winner checking part.

The most straightforward and the most violent way to check the winner is to scan the whole chess board and looking for a continuous 5 chesses, whether it is in a row, a column or a diagonal.

What is the problem of scanning the whole chess board? The most fatal problem is the poor performance. Although modern computers doesn't need this type of small optimization, but a huge performance problem is heaped up by small performance problems. So in this part, I decided to develop a new method of checking the winner, the answer is *local checking*.

The core logic of *local checking* is to choose a start point, and then check the 4 directions (diagonal, positive row, negative row, positive column, negative column) of the start point.

In this script, the start point will be the current point, which is the `current_player` selected where are their chesses were placed. Then, the function will start to check the 4 directions, which is turned into 4 tuples in the script.

```
def check_win(board, rowc, column, player):
    directions = [(0, 1), (1, 0), (1, 1), (1, -1)]
```

After that, the script will call a `for` function, use `change_r` and `change_c` as the variables, which stand for *changing row number* and *changing column number*, while iterates over the `directions` sequences, to check every possible ways of the directions' combinations.

Then, it will initialize the counter, or in a another way, set `count` equals to 1. I set it as 1 not 0, because the chess placed is counted as 1 chess in the 5 chesses to win.

```
    for change_r, change_c in directions:
        count = 1
```

Additionally, it will do a progress called *forward search*. Due to its actions - taking forward steps towards (change_c and change_r) until the changed value gets out of the range, or the current block was empty or the placed chess was not placed by current player.

```
        new_r, new_c = rowc + change_r, column + change_c
        while 0 <= new_r < SIZE and 0 <= new_c < SIZE and \
            board[new_r][new_c] == player:
            count += 1
            new_r += change_r
            new_c += change_c
```

Besides that, the script will then do the same thing again, but with opposite direction.

```
        new_r, new_c = rowc - change_r, column - change_c
        while 0 <= new_r < SIZE and 0 <= new_c < SIZE and \
```

```
        board[new_r][new_c] == player:
        count += 1
        new_r -= change_r
        new_c -= change_c
```

For the final piece of the entire function, the script will call `if` to do a simple comparison. If the counter's value is larger(not likely to happen unless the script was modded) or equals to 5 , then it will return a `True` signal, to tell the script's `main` function, that `current_player` has won. In opposite way, it will return `False` and let the script continue.

```
    if count >= 5:
        return True
return False
```

Section 4 : Main Loop

After the defining the module, the most crucial part of the script is the main loop. During the main loop, the script will call every function we defined earlier, to make a complete game experience.

At the very beginning, the main loop will call `create_board` function, to make a 2D array for the date storing of the chess board. Then it will initialize the `current_player` and `status_msg` variables, for the future use.

```
def main():
    board = create_board()
    current_player = BLACK
    status_msg = f"Player {current_player} please place."
```

After that, the main loop will call `while` function, to print a board and collect the user's input for the chess's coordinate, and initialize `result` and `error_err` variables through `parse_input` function, to recognition whether the user's input is legal. If user's input is legal, then turn it to the coordinate information used for placing the chess, or just simply terminate the script. After validating the user's input, the script will simply throws out a error message or continue the script and then refresh the script (call `print_board` function).

```
if result == 'QUIT':
    print("Interrupt signal detected.")
    break

if result is None:
    status_msg = f"Error: {error_err}"
    continue

row, col = result

if board[row][col] != EMPTY:
    status_msg = \
    f"Error: Position {user_input.upper()} is already taken."
    continue

board[row][col] = current_player
```

If the returned value for `check_win` function is `True`, then terminate the script and throws out a message, claiming who is the winner.

```
if check_win(board, row, col, current_player):
    print_board(board, f" Congrats! Player {current_player} wins!",\
    current_player)
    print("GAME OVER")
    break
```

If the chess board is filled with chesses entirely, then the script will terminate, but throws out a different message, indicates that the match is a draw.

```
if all(cell != EMPTY for row in board for cell in row):
    print_board(board, "It is a draw!", current_player)
    break
```

At the end of the main loop, the script will held the logic for the script to keep running, basically re-printing the `status_msg` and switching `current_player` after refreshing the chess board.

```
    status_msg = f"Player {current_player} placed at {user_input.upper()}"
    current_player = WHITE if current_player == BLACK else BLACK
```

Although the main loop has ended here, there are still extra commands to be operated. The main function of this part of code is to make sure that when the script was executed directly from the terminal, the main loop will be called and the functions will be defined, to prevent the script accidentally output something unexpected when the script file itself was imported as a module.

This operation improves the reusability and the portability of the script, even in other software platforms or hardware architecture.

```
if __name__ == "__main__":
  main()
```

Section 5 : Future Development Plan

In this period of this technical document, I will explain the future development plan of this script.

1. Adding the function of PvE. PvE is now a more popular way of training yourself. Based on this, I will adding PvE support for this Gomoku script, through AI technology, but not LLM, due to its weak ability in specific area, like playing chess and large performance cost. The potential solution will be Minimax model and Rapfi model.

Minimax model is a symbol of Symbolism AI, its basic principle is through mathematical functions and tree searching with depth.

Rapfi model is currently the most powerful open source AI models that represents the area of Connectionism AI, which based on CNN(Convolutional Neural Network).Representing the top ability of playing Gomoku game.

2. Adding more user-friendly UI. For example, a welcome screen is indeed a good way for improving the experience.

However, due to the need of `pygame` 3rd-party library, this script will not likely to add graphical interfaces, indicating that this script will keep terminal-based. Users will need a fully functional terminal emulator to gain the full experience. Otherwise, some features like automatically clearing and refreshing the board will not be functional.

Section 6 : Reusability and Portability

This script is written by *Python 3.14.3*, due to its cross-platform feature, it can run its basic function on any operating system that is supported by *Python* interpreter (>= 3.14.3).

This script can be imported as a module, pointing to a larger loop of another script.

This script has been tested on *Arch Linux x86_64* with *linux 6.19.9-arch1-1 kernel* on *Kitty* and *Windows 11 25H2* on *Windows Terminal*. Further testing is processing on *Chimera Linux x86_64* with *musl C library* and *aarch64* hardware architecture.

Section 7 : References

1. Python 3.14.3 Tutorial, available at https://docs.python.org/3/tutorial/index.html.
2. Windows Terminal Customize Actions, available at https://learn.microsoft.com/zh-tw/windows/terminal/customize-settings/actions.
3. Rapfi main page, available at https://github.com/dhbloo/rapfi.
4. Markdown Tutorial, available at https://markdown.com.cn/.

Section 8 : Redistribution and Availability

This project is distributed under MIT license.

Project source code and technical document copy are available at https://github.com/sarosaka-orz/HKDSE-ICT-SBA-Gomoku.